DACEFyN

Departamento Académico de Ciencias Exactas, Físicas y Naturales

UNIDAD 2: ALGORITMOS FUNDAMENTALES

Backend de Plataforma de Streaming de Musica

Integrante 1: _____

Legajo: _____

Integrante 2: _____

Legajo: _____

Integrante 3: _____

Legajo: _____

Fecha de entrega: _____

Indice

(Click derecho > Actualizar campo para refrescar numeros de pagina)

1. Contexto **3**

2. Requisitos Funcionales **3**

2.1 Modelo de Datos 3

2.2 Operaciones con Streams API 4

2.3 Algoritmos de Búsqueda y Ordenamiento 4

2.4 Patron Strategy 5

2.5 Concurrencia 5

2.6 API REST Basic 6

3. Restricciones Técnicas **6**

4. Entregables **7**

5. Criterios de Evaluación **7**

6. Información de Entrega **8**

OBJETIVO EDUCATIVO

Dominar algoritmos de búsqueda, ordenamiento y procesamiento funcional de colecciones mediante Streams API, aplicar patrones de diseño Strategy e introducir conceptos de concurrencia y desarrollo de APIs REST.

1. CONTEXTO

Desarrollar el backend de una plataforma tipo Spotify que permita gestionar canciones, crear playlists automáticas y generar recomendaciones, aplicando programación funcional y concurrente.

2. REQUISITOS FUNCIONALES

2.1 Modelo de Datos

Clase Cancion:

0. id: UUID (String)
1. titulo: String
2. artista: String
3. album: String
4. género: String (ENUM preferible: ROCK, POP, JAZZ, ELECTRONICA, CLASICA)
5. duracionSegundos: int
6. Reproducciones: AtomicInteger (para concurrencia)
7. rating: double (0.0 - 5.0)
8. fechaLanzamiento: LocalDate

2.2 Operaciones con Streams API (30%)

- ◻ Filtrado compuesto: por género, artista, rango de años, rating mínimo
- ◻ Top 10 mas reproducidas: sorted(Comparator) + limit(10) + collect

Estadisticas:

9. promedio de duración por género (Collectors.averagingInt)
10. artista más popular (maxBy reproducciones)
11. distribución por décadas (Collectors.groupingBy)

- ◻ Playlist automática: seleccionar canciones que sumen exactamente X minutos (aproximación al problema de la mochila usando recursión y Streams)

2.3 Algoritmos de Búsqueda y Ordenamiento (20%)

- ☐ Búsqueda binaria por título: lista pre-ordenada con Comparator.naturalOrder()
- ☐ Ordenamiento personalizado: Comparator.comparing(Cancion::getArtista).thenComparing(Cancion::getFechaLanzamiento).reversed()
- ☐ Búsqueda lineal con predicados múltiples (género AND año > X AND rating > Y)

2.4 Patron Strategy (20%)

Interfaz Estrategia Recomendación:

```
List<Cancion> recomendar(List<Cancion> catalogo, Cancion base)
```

Implementaciones:

- ☐ RecomendaciónPorGénero: mismo género, ordenadas por rating
- ☐ RecomendaciónPorPopularidad: top 5 más reproducidas global
- ☐ RecomendacionDescubrimiento: menos de 1000 reproducciones, fecha reciente (< 2 años), género diferente al habitual

2.5 Concurrencia (20%)

- ☐ Simular 10 usuarios reproduciendo simultáneamente usando ExecutorService
- ☐ ThreadPool con 5 threads fijos
- ☐ Cada 'reproducción' incrementa contador atomicamente
- ☐ Medir tiempo total de procesamiento vs. secuencial

2.6 API REST Básica con Spring Boot (10%)

CancionController con:

12. GET /api/canciones - listar todas (con paginación opcional)
13. GET /api/canciones/{id} - buscar por ID
14. GET /api/canciones/buscar?titulo=xxx&artista=yyy - búsqueda filtrada
15. POST /api/canciones/{id}/reproducir - incrementar contador

- ☐ Uso de ResponseEntity para códigos HTTP apropiados

3. RESTRICCIONES TÉCNICAS

16. Java 17+, Spring Boot 3.x
17. Base de datos H2 en memoria (opcional) o ArrayList para simplificar
18. Postman o curl para probar endpoints REST
19. Maven con dependencias: spring-boot-starter-web, lombok (opcional)

4. ENTREGABLES

1. Código fuente completo (proyecto Maven/Gradle)

2. Colección de Postman con 5 requests de prueba

3. Análisis de complejidad Big O para:

20. Búsqueda binaria implementada
21. Algoritmo de playlist automática

- ☐ Captura de ejecución concurrente mostrando incremento de reproducciones

5. CRITERIOS DE EVALUACIÓN

Aspecto	Peso
Uso avanzado de Streams API	25%
Implementacion de Strategy	20%
Algoritmos de busqueda/ordenamiento	20%
Concurrencia correcta	20%
API REST funcional	10%
Análisis de complejidad	5%

6. INFORMACIÓN DE ENTREGA

Fecha de entrega: [Indicar fecha]

Modalidad: Grupos de 2-3 estudiantes